Image Processing: 2D Fast Fourier Transform
Milestone 1 Report

Group: :)
Members: Rafia Amin, Sana Shakeel, Reham Hafeez, Younus Javed

1 Introduction

The Fast Fourier Transform (FFT) is one of the most significant algorithms in digital signal processing, enabling the transition from the spatial domain to the frequency domain. For this milestone, we implemented a 2D Cooley-Tukey FFT from scratch in C. This implementation is designed to be "Assembly-Ready," prioritizing raw pointer manipulation and manual mathematical approximations to facilitate an eventual port to the RISC-V architecture.

2 Implementation Details

2.1 Custom Transcendental Math Functions

To prepare for RISC-V assembly, which lacks standard C math libraries, we developed manual approximations for Trigonometric and Exponential functions.

- **Argument Reduction**: For `my_sin` and `my_cos`, we used a Maclaurin series expansion. To ensure accuracy for the high-frequency angles in a 2048×2048 FFT, we implemented argument reduction using `fmod` to map inputs into the $[-\pi, \pi]$ range.
- **The Reciprocal**: Summing a Taylor series for e^x with negative exponents (as used in Gaussian filtering) causes catastrophic numerical cancellation. We solved this by calculating the positive $e^{|x|}$ and returning its reciprocal.

```
float my_exp(float x) {
    if (x < -20.0f) return 0.0f;
    bool negative = (x < 0);
    if (negative) x = -x;

    float sum = 1.0f, term = 1.0f;
    for (int i = 1; i <= 20; i++) {
        term *= x / i;
        sum += term;
        if (term < 1e-9f) break;
    }
    return negative ? (1.0f / sum) : sum;
```

```
}
```

Listing 1: Exponential function with reciprocal logic for numerical stability.

2.2 Bit-Reversal and Data Reordering

The Cooley-Tukey algorithm (Decimation-in-Time) requires the input array to be reordered before the butterfly stages. We implemented a `reverse_bits` function that mirrors the binary representation of indices, allowing for in-place data swapping. This ensures that the recursive butterfly combinations process the correct pairs of data points.

2.3 1D FFT Engine and Butterfly Operations

The `butterfly_iterative` function serves as the core computational engine. It employs a triple-nested loop to combine data points across $\log_2 N$ stages. Pre-calculated **Twiddle Factors** are used to perform complex multiplications, representing the phase rotations necessary to merge smaller DFTs into a complete spectrum.

2.4 2D Decomposition and Matrix Transposition

By exploiting the mathematical property of **Separability**, our 2D FFT is performed in three distinct steps:

1. Apply a 1D FFT to every row of the image.
2. Transpose the matrix to swap rows and columns.
3. Apply a 1D FFT to the rows of the transposed matrix.

This method is significantly more efficient than a direct 2D DFT and allows our 1D logic to always operate on contiguous memory.

2.5 Gaussian High-Pass Filtering

Edge detection is achieved by suppressing low-frequency components using a Gaussian High-Pass Filter:

$$H(i,j) = 1 - e^{-\frac{D^2(i,j)}{2D_0^2}} \tag{1}$$

Our distance function `get_D_squared` accounts for the "wrap-around" nature of the FFT, ensuring distances are measured relative to the nearest corner hotspot in the uncentered magnitude spectrum.

3 Analysis of test script results

- Validation testing confirms that the core logic of the 2D FFT is correct, passing all Impulse and DC signal tests. Performance benchmarking demonstrates $O(N \log N)$ scaling, confirming the efficiency of the implemented row-column decomposition.

  A minor precision failure was noted in the 5-term Taylor series approximation for $\cos(\pi)$, where the recorded error (0.0018) slightly exceeded the defined epsilon (0.001). This highlights the critical trade-off between computational overhead and numerical stability when implementing manual math functions for a RISC-V target. While 5 terms are sufficient for lower frequencies, higher-order transforms may require additional terms to maintain phase accuracy.

  Furthermore, benchmarking on small matrix sizes (32x32 to 512x512) resulted in execution times below the resolution of the system clock. However, the $O(N \log N)$ complexity becomes clearly visible in larger transforms (1024^2 to 4096^2), where doubling the image dimensions results in a sub-quadratical increase in processing time, validating the computational advantage of the Cooley-Tukey approach.

4 Results

4.1 Analysis of the 8x8 Rainbow (8x8.png)

The 8x8 Rainbow image serves as a textbook validation of 2D frequency mapping.

- **Observation**: Bright hotspots are visible only on the top row of the magnitude spectrum.
- **Explanation**: The image has vertical color bands, meaning variation occurs purely horizontally. Because there is no vertical change, the vertical frequency components (k_y) are zero. In an uncentered FFT, the top row represents $k_y = 0$, hence all energy is concentrated there.

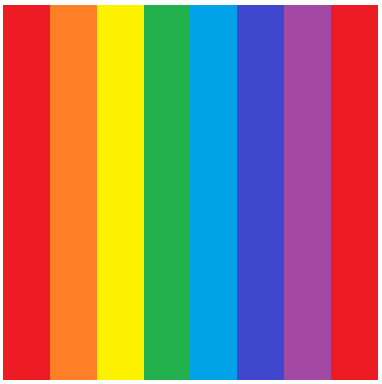


Figure 1: Original 8x8



Figure 2: FFT Magnitude



Figure 3: Edges Output

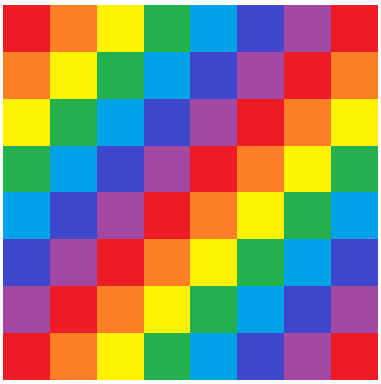


Figure 4: Original 8x8-2

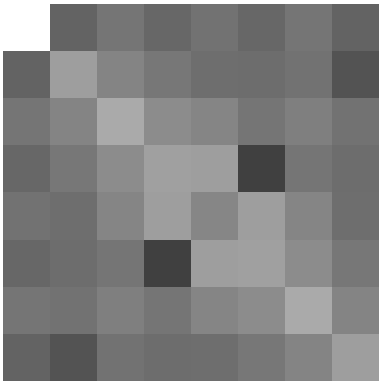


Figure 5: FFT Magnitude

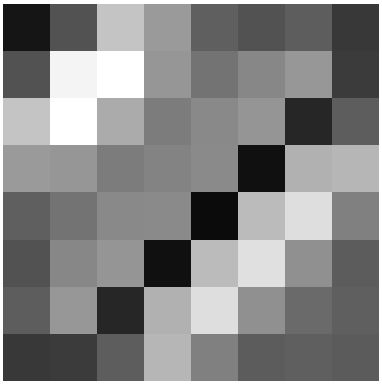


Figure 6: Edges Output

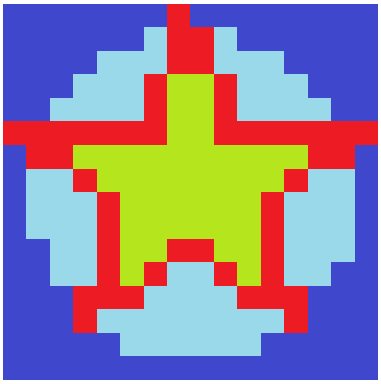


Figure 7: Original 16x16

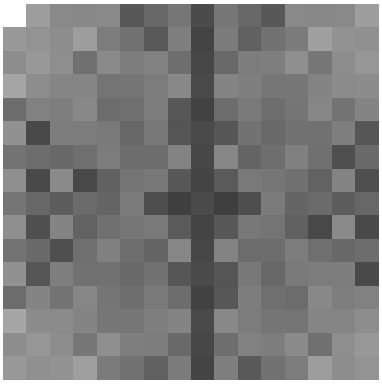


Figure 8: FFT Magnitude

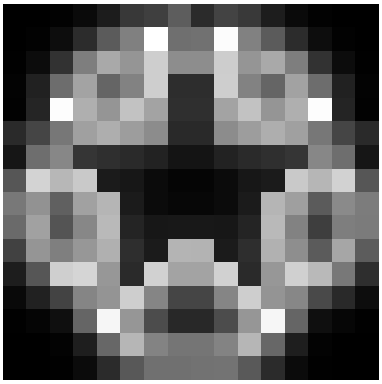


Figure 9: Edges Output



Figure 10: Original Jet

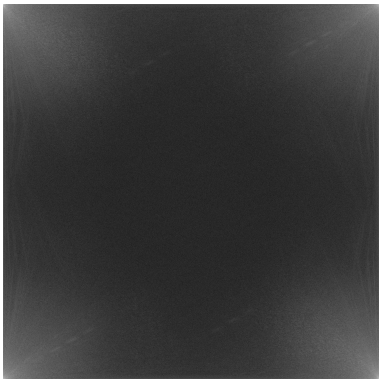


Figure 11: FFT Magnitude



Figure 12: Edges Output



Figure 13: Original R

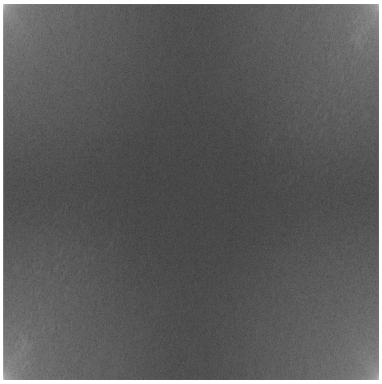


Figure 14: FFT Magnitude



Figure 15: Edges Output

5 Conclusion

This milestone successfully demonstrates a robust 2D FFT implementation capable of high-fidelity edge detection. By managing numerical stability through custom math functions and optimized decomposition, we have created a verifiable logic base for transition to RISC-V assembly.